

HANDSMEN THREADS

Automation, Apex & Asynchronous Processing

Salesforce CRM Project Documentation 3

Document Control

Item	Details
Project	HandsMen Threads – Salesforce CRM
Business Domain	Premium men's fashion / bespoke tailoring
Platform	Salesforce
Document	03 – Automation, Apex & Asynchronous Processing
Module Group	Flows, Apex, Triggers and Batch Jobs
Purpose	Detailed module-wise implementation and verification evidence
Evidence	Completed Salesforce configuration screenshots supplied with the project documentation

1. Purpose

This document records the automation and programmatic processing implemented for HandsMen Threads. It covers the Skills Wallet modules related to Record-Triggered Flows, Apex, Apex Triggers, Asynchronous Apex, Batch Jobs and Scheduled Processing.

The objective is to show how the business requirements are converted into automated Salesforce behavior. The implementation combines declarative automation for record-driven events and notifications with programmatic Apex for custom processing and asynchronous execution.

The evidence in this document is taken from the completed Salesforce implementation. Each screenshot is placed next to the explanation of the functionality it demonstrates, so the reviewer can verify the module without relying only on a short caption.

2. Business Requirements Mapped to Automation

HandsMen Threads requires automation in four main operational areas. Each area has a business event or scheduled condition, a Salesforce processing mechanism and a resulting business action.

Requirement	Business Condition	Salesforce Implementation	Expected Outcome
Automated Order Confirmation	Order reaches configured confirmation state	Record-Triggered Flow + Order Confirmation Email Alert	Customer receives an order confirmation.

Dynamic Loyalty Program	Purchase/order information requires loyalty update	Flow decision/branching logic and configured update processing	Customer loyalty information is updated.
Proactive Stock Alert	Stock Quantity falls below five	Record-Triggered Flow + Stock Alert Email Alert	Warehouse/operations team receives a low-stock notification.
Scheduled Bulk Order Updates	Daily midnight processing	Batch / Scheduled Apex	Bulk-order financial and inventory processing is performed.

This mapping establishes traceability between the original HandsMen Threads use case and the Salesforce automation implemented in the org.

3. Automation Architecture

The solution uses a layered automation approach. Record-Triggered Flows react to record changes and evaluate business conditions. Email Alerts provide standardized communication. Apex and Triggers handle custom programmatic processing, while Batch and Scheduled Apex support background and time-based processing.

Layer	Salesforce Feature	Role in HandsMen Threads
Event-driven automation	Record-Triggered Flow	Starts processing when relevant records change.
Decision logic	Flow Decision elements	Selects the correct business path.
Communication	Email Alerts	Invokes configured customer/warehouse notifications.
Programmatic logic	Apex / Apex Trigger	Executes custom business logic in response to record events.
Bulk processing	Batch Apex	Processes bulk-order records in manageable asynchronous batches.
Time-based processing	Scheduled Apex	Starts the daily bulk-order process at the configured time.

4. Record-Triggered Flows

A Record-Triggered Flow starts automatically when a Salesforce record is created or updated and the configured entry conditions are satisfied. Flow Builder allows the project to represent business logic visually through start conditions, decisions, branches and actions.

For HandsMen Threads, Flow is appropriate for business processes where the system needs to react immediately to a record change and then follow a clearly defined business rule. This reduces repetitive manual work and makes the process consistent for every qualifying record.

4.1 Order Confirmation Automation

The Order Confirmation process is designed to respond when an order reaches the configured confirmation state. The Flow evaluates the order and follows the appropriate path to the Order Confirmation Email Alert. The important

implementation point is that the Flow is not merely storing an order status. It connects the order-state event to a customer communication action. Once the configured condition is satisfied, the email action can use the standardized template documented in Document 2.

4.1.1 Processing Sequence

- An order record is created or updated.
- The Record-Triggered Flow evaluates the configured entry conditions.
- The Flow checks the relevant order condition/status through its decision logic.
- The appropriate branch is selected.
- The Order Confirmation Email Alert action is executed.
- The configured order-confirmation message is sent to the intended recipient.

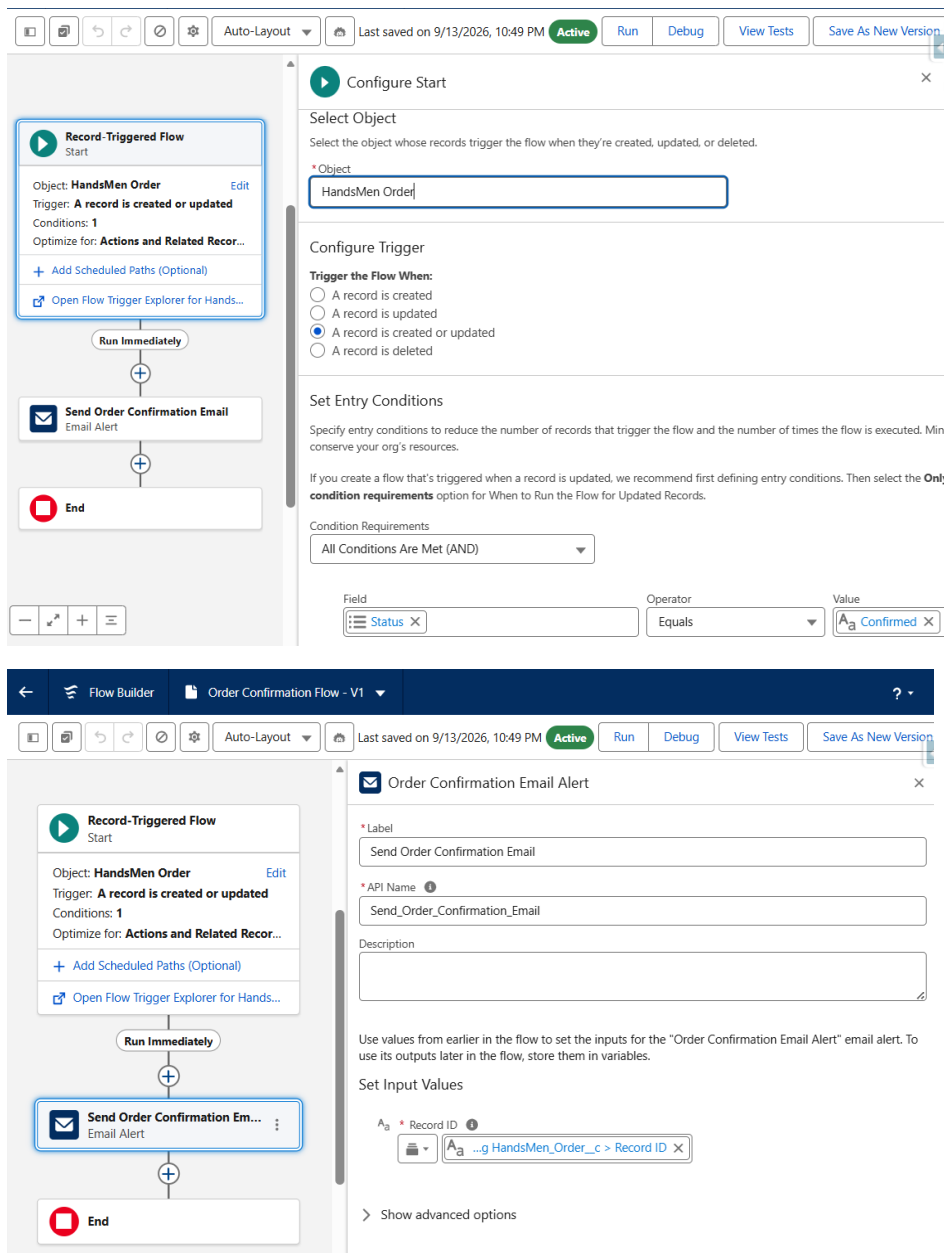


Figure – Combined Flow and Email Alert evidence showing the order-confirmation automation and email-action configuration.

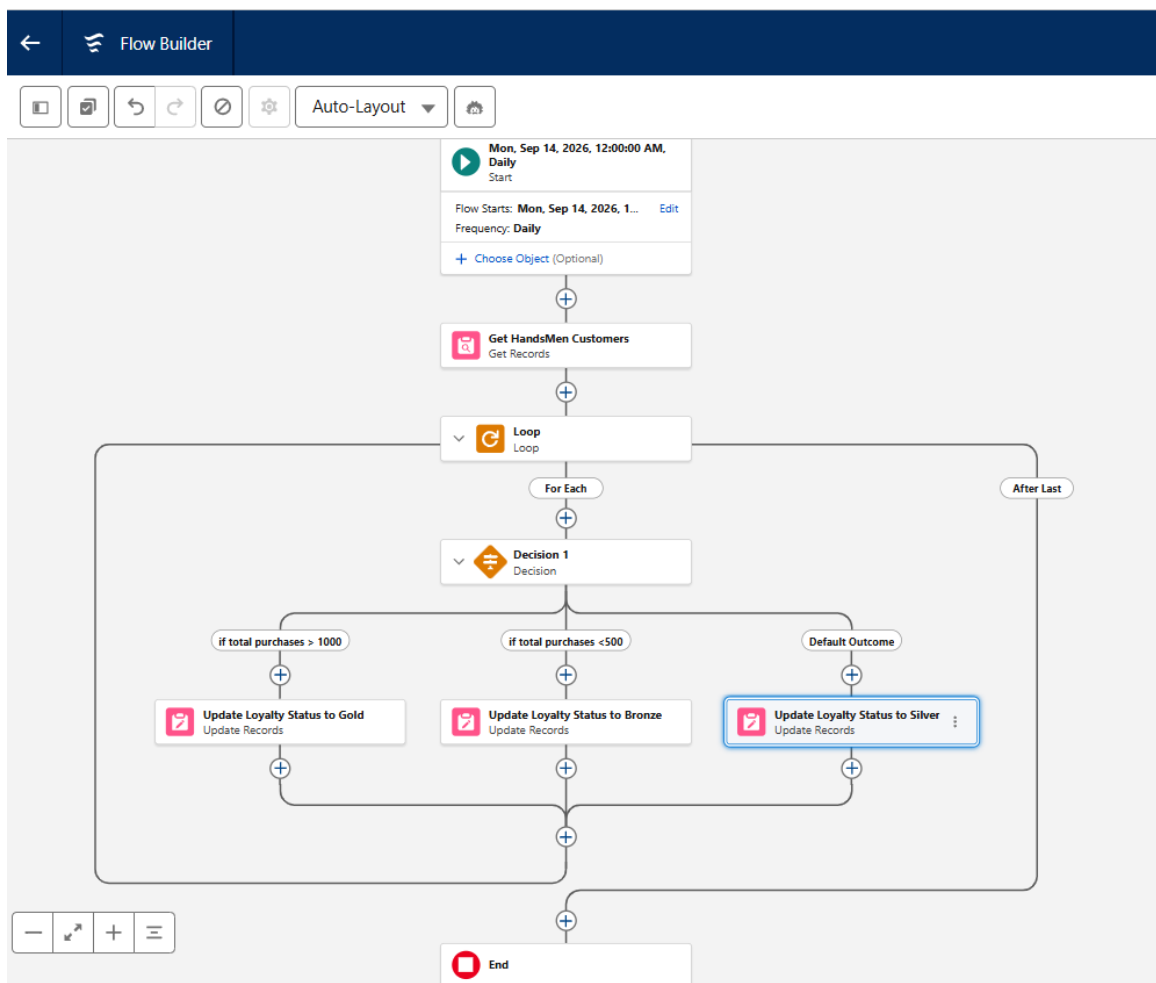
4.2 Loyalty Program Automation

The Loyalty Program automation is intended to maintain customer loyalty information using purchase or order-related information. The supplied evidence shows Flow Builder decision/branching logic together with related programmatic implementation evidence.

The Flow uses decision logic to determine which path should be followed. The actual loyalty thresholds and field values must correspond to the configuration present in the completed Salesforce org.

4.2.1 Business Processing

- Relevant purchase or order information becomes available.
- The automation evaluates the configured loyalty criteria.
- The decision element determines the applicable branch.
- The customer loyalty information is updated according to the configured logic.
- The resulting customer information can support the loyalty communication process.



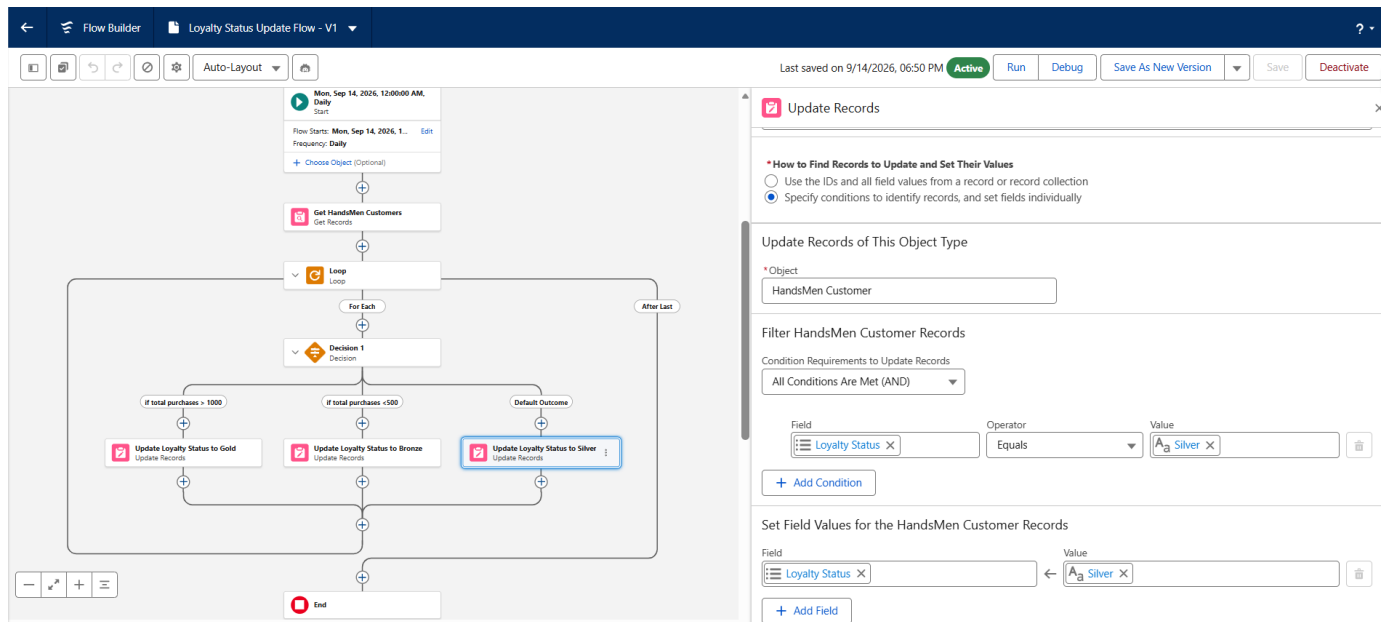


Figure – Flow Builder decision/branching evidence for the loyalty-related automation, together with the related Apex/trigger implementation evidence.

4.3 Proactive Stock Alert Automation

The Proactive Stock Alert process implements one of the clearest business rules in the HandsMen Threads use case: when inventory stock falls below five units, the warehouse/operations team should be notified.

The automation evaluates the Inventory record and uses a decision path to determine whether the low-stock condition is true. When the condition is met, the Flow invokes the configured Stock Alert Email Alert.

4.3.1 Stock Threshold Rule

Condition	Automation Action	Business Result
Stock Quantity < 5	Invoke Stock Alert Email Alert	Warehouse/operations team is notified so restocking can be considered.

4.3.2 Processing Sequence

- Inventory stock quantity is created or updated.
- The Record-Triggered Flow evaluates the inventory record.
- The decision element checks the configured stock threshold.
- If Stock Quantity is below five, the low-stock branch is selected.
- The Stock Alert Email Alert action is invoked.
- The configured warehouse/operations recipient receives the notification.

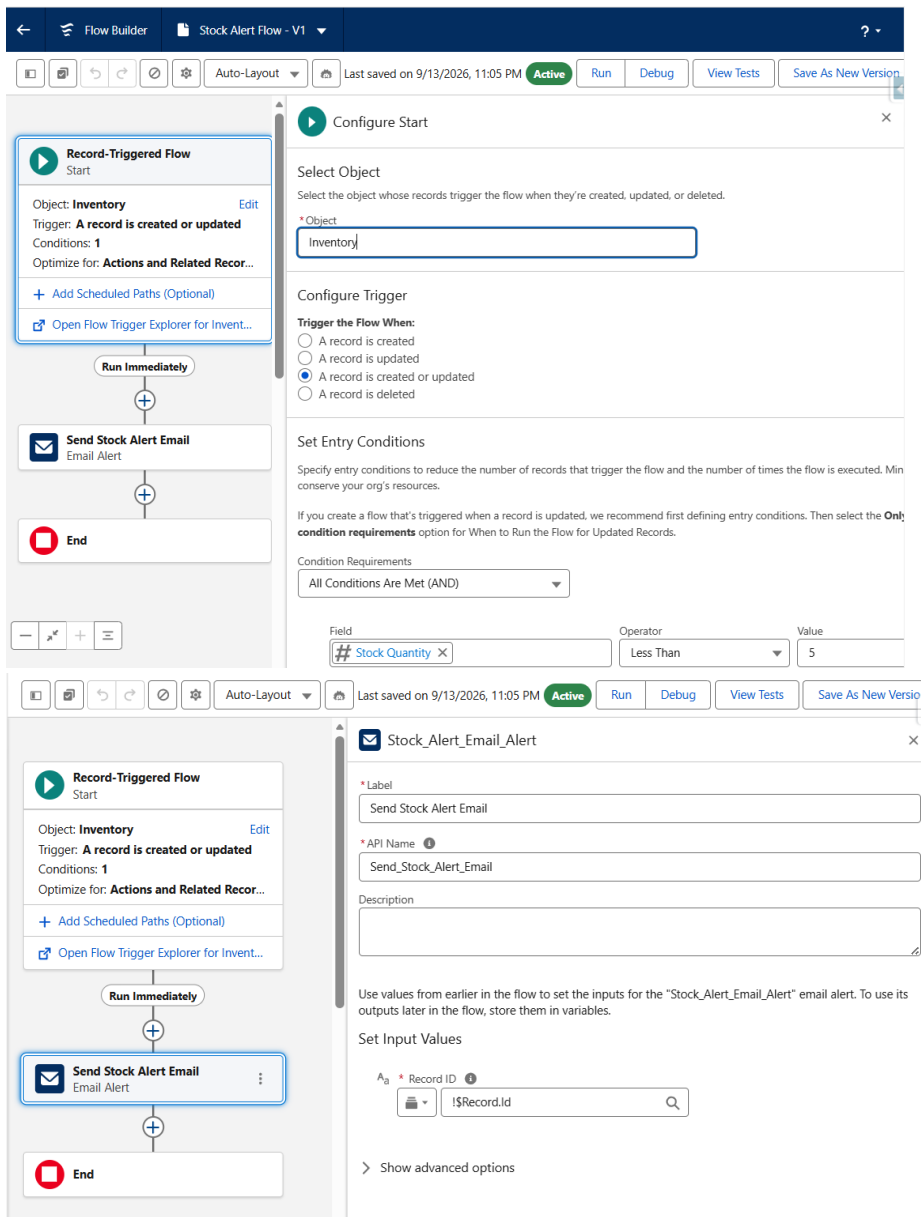


Figure – Flow Builder decision/branching evidence used to verify the stock-alert business rule.

5. Automation Using Apex

Apex is Salesforce's server-side programming language and is used when business requirements require custom logic beyond declarative configuration. The supplied evidence contains Apex code displayed in the Salesforce Developer Console.

In HandsMen Threads, the Apex implementation forms the programmatic layer that can perform custom calculations, query related records, apply updates and support asynchronous processing where required.

5.1 Apex Implementation Structure

The implementation should be reviewed as two connected parts: the Apex class contains the reusable business logic, while the Apex Trigger provides the event-driven entry point that invokes that logic when the configured object event occurs.

5.2 Apex Processing Responsibilities

- Receive the relevant Salesforce record context.
- Evaluate the configured business condition.
- Query related records when required.
- Perform the required business calculation or processing.
- Apply record changes using appropriate DML operations.
- Return or complete the processing without requiring manual intervention.
- Support testable and maintainable business logic.

5.3 Apex Evidence Interpretation

The supplied Apex screenshot shows the programmatic implementation in Developer Console. The code evidence should be checked against the actual class and trigger names in the Salesforce org. Exact names are intentionally left as fields to complete from the active org rather than being guessed from a low-resolution screenshot.

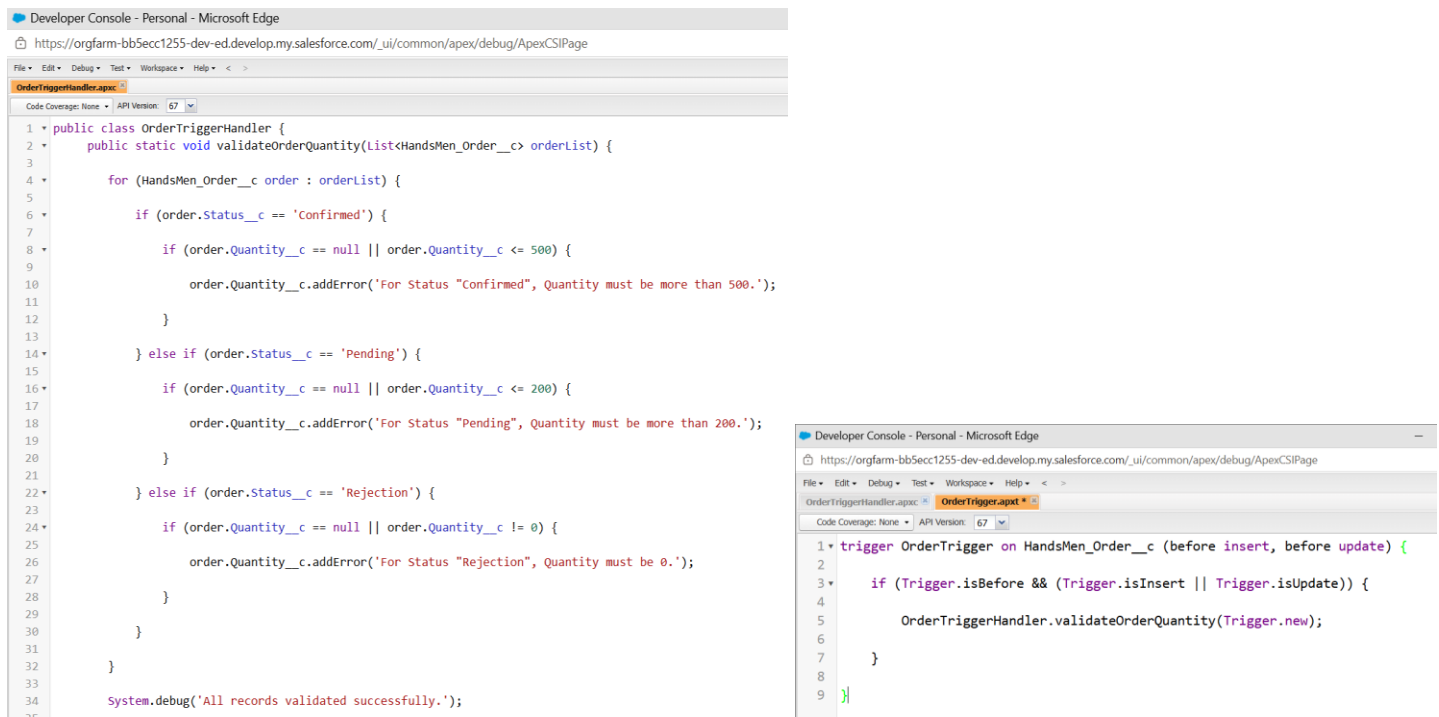


Figure – Apex code and trigger evidence included with the loyalty/programmatic implementation.

6. Apex Trigger Design

An Apex Trigger is an event-driven mechanism that executes when a configured operation occurs on a Salesforce object. It determines when the programmatic logic should be invoked, while the supporting Apex class determines what business processing should take place.

For HandsMen Threads, the trigger should therefore be understood as the connection between the Salesforce data event and the custom Apex business logic.

7. Asynchronous Apex

Asynchronous Apex is used when processing is better performed outside the immediate user transaction. It is particularly useful for operations involving multiple records, bulk workloads or scheduled execution.

The HandsMen Threads use case identifies daily bulk-order processing as the principal asynchronous requirement. The asynchronous design separates background processing from the immediate user interaction and allows the system to process operational work at the configured schedule.

7.1 Benefits for HandsMen Threads

- Supports processing of multiple bulk-order records.
- Reduces dependency on manual execution.
- Allows background processing to occur at a scheduled time.
- Provides Salesforce job information for monitoring.
- Helps separate scheduled operational work from immediate UI transactions.

7.2 Asynchronous Processing Flow

Stage	Processing
1	Scheduled execution starts the configured asynchronous process.
2	Batch Apex identifies records that satisfy the selection logic.
3	Records are processed in manageable groups.
4	Required financial and inventory updates are applied.
5	Completion information becomes available for monitoring.

8. Batch Jobs

Batch Apex is designed for processing large volumes of Salesforce records in manageable groups. In HandsMen Threads, the batch process supports the daily bulk-order requirement and is intended to apply the required financial and inventory updates.

The Batch implementation should be reviewed together with its scheduling configuration. Batch Apex defines how the records are processed, while Scheduled Apex determines when the process begins.

8.1 Batch Processing Responsibilities

- Select the records that qualify for bulk-order processing.
- Process the selected records in batches.
- Apply the required financial updates.
- Adjust the relevant inventory information.
- Complete the asynchronous job.
- Provide execution information that can be reviewed after processing.

8.2 Batch Evidence

The supplied Developer Console evidence includes the batch-related Apex implementation and execution context. The reviewer should verify the class name, selection logic and processing behavior against the actual Salesforce implementation.

```

1  global class InventoryBatchJob implements Database.Batchable<SObject>, Schedulable {
2
3  *   global Database.QueryLocator start(Database.BatchableContext BC) {
4
5      return Database.getQueryLocator(
6
7          'SELECT Id, Stock_Quantity__c FROM Product__c WHERE Stock_Quantity__c < 10'
8
9      );
10
11  }
12
13  *   global void execute(Database.BatchableContext BC, List<SObject> records) {
14
15      List<HandsMen_Product__c> productsToUpdate = new List<HandsMen_Product__c>();
16
17      // Cast SObject list to Product__c list
18
19  *   for (SObject record : records) {
20
21      HandsMen_Product__c product = (HandsMen_Product__c) record;
22
23      product.Stock_Quantity__c += 50; // Restock logic
24
25      productsToUpdate.add(product);
26
27  }
28
29  *   if (!productsToUpdate.isEmpty()) {
30
31  *       try {
32
33           update productsToUpdate;
34
35  *       } catch (DmlException e) {
36
37           System.debug('Error updating inventory: ' + e.getMessage());
38
39       }
40
41   }
42
43   }
44
45  *   global void finish(Database.BatchableContext BC) {
46
47       System.debug('Inventory Sync Completed');
48
49   }
50
51   // Scheduler Method
52
53  *   global void execute(SchedulableContext SC) {
54
55       InventoryBatchJob batchJob = new InventoryBatchJob();
56
57       Database.executeBatch(batchJob, 200);
58
59   }
60
61 }

```

Figure – Apex implementation, execution/debug information and scheduled-job evidence associated with the asynchronous processing implementation.

9. Scheduled Processing

Scheduled Apex allows a Salesforce Apex process to be executed at a specified time without requiring a user to start it manually. HandsMen Threads uses this mechanism to support the requirement for daily midnight bulk-order processing.

The supplied evidence includes scheduling code and the Salesforce Scheduled Jobs interface. The code provides the scheduled execution definition, while the Scheduled Jobs page provides administrative visibility into the configured job.

9.1 Daily Midnight Processing

Scheduling Requirement	Implementation
Frequency	Daily
Required Time	Midnight

Process	Bulk-order processing
Execution Mechanism	Scheduled Apex leading to asynchronous/batch processing
Business Purpose	Update financial records and adjust inventory for daily operations

9.2 Scheduled Job Verification

- Open Setup → Scheduled Jobs.
- Locate the HandsMen Threads scheduled job.
- Verify the associated Apex class.
- Verify the configured execution schedule.
- Confirm that the job is listed as a scheduled process.
- Review the job after execution to confirm that the expected background processing occurred.

The image displays two screenshots from a Salesforce environment. The top screenshot shows the Developer Console with an anonymous Apex class named `InventoryBatchJob` scheduled to run daily at 11:00 AM. The code is as follows:

```
1 System.schedule('Daily Inventory Sync', '0 0 0 * * ?', new InventoryBatchJob());
2
```

The bottom screenshot shows the 'Scheduled Jobs' page in the Salesforce Setup menu. It lists the following job:

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Inventory Sync	Mudugu_Devi Durga Sudhishna	9/14/2026, 6:47 AM		9/15/2026, 12:00 AM	Scheduled Apex	08ef00000aeeEW
Del	Loyalty_Status_Update_Flow-1	Mudugu_Devi Durga Sudhishna	9/14/2026, 6:20 AM		9/15/2026, 12:00 AM	Scheduled Flow	08ef00000aee7b7

Figure – Scheduled processing code and Salesforce Scheduled Jobs evidence showing the configured background execution.

10. End-to-End Automation Sequence

The complete HandsMen Threads automation sequence connects record changes, decision logic, notifications and asynchronous processing.

Step	System Activity	Purpose
1	Order or Inventory record is created or updated.	Captures the business event.
2	Record-Triggered Flow or Apex Trigger evaluates the configured condition.	Determines which business rule applies.
3	Flow decision or Apex logic selects the processing path.	Routes the record to the required action.
4	Email Alert is invoked where communication is required.	Sends the configured customer/warehouse message.
5	Scheduled/Batch processing handles bulk-order work.	Performs background financial and inventory processing.
6	Records and job information are reviewed.	Confirms expected results.

This architecture demonstrates how the HandsMen Threads project combines declarative and programmatic automation instead of relying on a single automation mechanism.

11. Automation Test Cases

Test ID	Feature	Input / Action	Verified Result
TC-AUTO-01	Order Confirmation Flow	Move an order to the configured confirmation state.	Order Confirmation Email Alert is invoked.
TC-AUTO-02	Loyalty Automation	Perform the configured purchase-history/order change.	Loyalty information updates according to configured logic.
TC-AUTO-03	Stock Alert	Set Inventory Stock Quantity below 5.	Stock Alert Email Alert is invoked.
TC-AUTO-04	Apex / Trigger	Perform the event handled by the trigger.	Configured Apex logic executes successfully.
TC-AUTO-05	Batch Job	Run the batch with representative bulk-order records.	Required financial/inventory updates are applied.
TC-AUTO-06	Scheduled Job	Review scheduled job configuration/execution.	Job is scheduled for intended daily processing.

12. Debugging & Monitoring

Automation should be monitored after configuration and during testing. Flow Builder debugging can be used to inspect decision outcomes and actions. Apex debug logs can help trace programmatic execution, SOQL queries, DML operations and exceptions. Scheduled Jobs and asynchronous job information provide visibility into background execution.

12.1 Monitoring Checklist

- Confirm that the relevant Flow is activated.
- Use Flow debugging to inspect decision outcomes and actions.
- Review Apex debug logs when programmatic behavior needs investigation.
- Check for query, DML and exception errors.
- Review Scheduled Jobs for the intended schedule.
- Review asynchronous job information after Batch Apex execution.

13. Deployment & Maintenance Considerations

Before production deployment, all Flow, Apex, Trigger, Batch and Scheduled Apex components should be validated in the target environment. The project should also be tested using representative records before enabling automated processing for production users.

- Confirm required Flows are active.
- Verify Email Alerts reference the correct templates and recipients.
- Review Apex and Trigger behavior against the final data model.
- Run appropriate Apex tests before deployment.
- Verify Batch record selection and processing logic.
- Verify the scheduled execution time and associated Apex class.
- Review automation errors and asynchronous job results.
- Monitor important record changes after deployment.

14. Evidence Interpretation

The evidence screenshots supplied with the project show the implemented automation at multiple levels. The first combined screenshot contains Flow Builder and Email Alert configurations for order and stock-related processing. The second combined screenshot shows the loyalty Flow decision structure together with Apex/Trigger code. The third screenshot contains Apex implementation and execution/debug information together with the scheduled-job context. The fourth screenshot shows the scheduling code and Salesforce Scheduled Jobs interface.

The screenshots are therefore used as verification evidence rather than as decorative images. The written sections explain what each configuration is intended to accomplish, while the screenshot allows the reviewer to inspect the corresponding Salesforce implementation.

Where the supplied screenshots do not provide an exact readable class name, trigger name, schedule expression or field value in the document text, a blank has been deliberately retained. Those values should be copied from the active Salesforce org rather than guessed.

15. Final Conclusion

HandsMen Threads demonstrates an end-to-end Salesforce automation architecture for a premium fashion and bespoke tailoring business. Record-Triggered Flows automate event-driven business processes, Email Alerts support customer and warehouse communication, Apex and Triggers provide custom programmatic processing, and Batch/Scheduled Apex support bulk and time-based processing.

The implementation reduces repetitive manual work, improves response to customer and inventory events, and provides a structured approach for processing bulk operational records. Together with the data-management and security/communication documents, this document completes the three-part Skills Wallet evidence package.